Lab8 orm拓展

本章主要介绍python使用orm对postgres数据库进行多表连接，子查询，反向查询等一些操作，并且进行综合练习

预先准备

连接数据库

In [3]:
```
import sqlalchemy
from sqlalchemy import Column, String, create_engine, Integer, Text, Date
from sqlalchemy.orm import sessionmaker,scoped_session
from sqlalchemy.ext.declarative import declarative_base
import time
from sqlalchemy import create_engine
engine = create_engine("postgresql://ecnu10245102456:ECNU10245102456@pgm-uf67ua6nah7b72h99o.r
    max_overflow=0,
    # 链接池大小
    pool_size=5,
    # 链接池中没有可用链接则最多等待的秒数，超过该秒数后报错
    pool_timeout=10,
    # 多久之后对链接池中的链接进行一次回收
    pool_recycle=1,
    # 查看原生语句（未格式化）
    echo=True
)
Session = sessionmaker(bind=engine)
session = scoped_session(Session)
DbSession = sessionmaker(bind=engine)
session = DbSession()
```

创建数据表

In [4]:
```
from sqlalchemy.ext.declarative import declarative_base
from sqlalchemy import Enum,ForeignKey,UniqueConstraint
from sqlalchemy.orm import relationship
Base = declarative_base()

class StudentsNumberInfo(Base):
    """学号表"""
    __tablename__ = "studentsNumberInfo"
    id = Column(Integer, primary_key=True, autoincrement=True, comment="主键")
    number = Column(Integer, nullable=False, unique=True, comment="学生编号")
    admission = Column(Date, nullable=False, comment="入学时间")
    graduation = Column(Date, nullable=False, comment="毕业时间")

class TeachersInfo(Base):
    """教师表"""
    __tablename__ = "teachersInfo"
    id = Column(Integer, primary_key=True, autoincrement=True, comment="主键")
    number = Column(Integer, nullable=False, unique=True, comment="教师编号")
    name = Column(String(64), nullable=False, comment="教师姓名")
    gender = Column(String(1), nullable=False, comment="教师性别")
    age = Column(Integer, nullable=False, comment="教师年龄")
class ClassesInfo(Base):
    """班级表"""
    __tablename__ = "classesInfo"
    id = Column(Integer, primary_key=True, autoincrement=True, comment="主键")
```

```python
    number = Column(Integer, nullable=False, unique=True, comment="班级编号")
    name = Column(String(64), nullable=False, unique=True, comment="班级名称")
    # 一对一关系必须为连接表的连接字段创建UNIQUE的约束，这样才能是一对一，否则是一对多
    fk_teacher_id = Column(
        Integer,
        ForeignKey(
            "teachersInfo.id",
            ondelete="CASCADE",
            onupdate="CASCADE",
        ),
        nullable=False,
        unique=True,
        comment="班级负责人"
    )
    # 下面这2个均属于逻辑字段，适用于正反向查询。在使用ORM的时候，我们不必每次都进行JOIN查询，
    # 这种逻辑字段不会在物理层面上创建，它只适用于查询，本身不占据任何数据库的空间
    # sqlalchemy的正反向概念与Django有所不同，Django是外键字段在那边，那边就作为正
    # 而sqlalchemy是relationship字段在那边，那边就作为正
    # 比如班级表拥有 relationship 字段，而老师表不曾拥有
    # 那么用班级表的这个relationship字段查老师时，就称为正向查询
    # 反之，如果用老师来查班级，就称为反向查询
    # 另外对于这个逻辑字段而言，根据不同的表关系，创建的位置也不一样：
    #  - 1 TO 1：建立在任意一方均可，查询频率高的一方最好
    #  - 1 TO M：建立在M的一方
    #  - M TO M：中间表中建立2个逻辑字段，这样任意一方都可以先反向，再正向拿到另一方
    #  - 遵循一个原则，ForeignKey建立在那个表上，那个表上就建立relationship
    #  - 有几个ForeignKey，就建立几个relationship
    # 总而言之，使用ORM与原生SQL最直观的区别就是正反向查询能带来更高的代码编写效率，也更加简单
    # 甚至我们可以不用外键约束，只创建这种逻辑字段，让表与表之间的耦合度更低，但是这样要避免脏
    # 班级负责人，这里是一对一关系，一个班级只有一个负责人
    leader_teacher = relationship(
        # 正向查询时所链接的表，当使用 classesInfo.leader_teacher 时，它将自动指向fk的那一条记
        "TeachersInfo",
        # 反向查询时所链接的表，当使用 teachersInfo.leader_class 时，它将自动指向该老师所管理的
        backref="leader_class",
    )
class ClassesAndTeachersRelationship(Base):
    """任教老师与班级的关系表"""
    __tablename__ = "classesAndTeachersRelationship"
    id = Column(Integer, primary_key=True, autoincrement=True, comment="主键")
    # 中间表中注意不要设置单列的UNIQUE约束，否则就会变为一对一
    fk_teacher_id = Column(
        Integer,
        ForeignKey(
            "teachersInfo.id",
            ondelete="CASCADE",
            onupdate="CASCADE",
        ),
        nullable=False,
        comment="教师记录"
    )
    fk_class_id = Column(
        Integer,
        ForeignKey(
            "classesInfo.id",
            ondelete="CASCADE",
            onupdate="CASCADE",
        ),
        nullable=False,
        comment="班级记录"
    )
    # 多对多关系的中间表必须使用联合唯一约束，防止出现重复数据
    __table_args__ = (
        UniqueConstraint("fk_teacher_id", "fk_class_id"),
    )
    # 逻辑字段
```

```python
    # 给班级用的，查看所有任教老师
    mid_to_teacher = relationship(
        "TeachersInfo",
        backref="mid",
    )
    # 给老师用的，查看所有任教班级
    mid_to_class = relationship(
        "ClassesInfo",
        backref="mid"
    )
class StudentsInfo(Base):
    """学生信息表"""
    __tablename__ = "studentsInfo"
    id = Column(Integer, primary_key=True, autoincrement=True, comment="主键")
    name = Column(String(64), nullable=False, comment="学生姓名")
    gender = Column(String(1), nullable=False, comment="学生性别")
    age = Column(Integer, nullable=False, comment="学生年龄")
    # 外键约束
    # 一对一关系必须为连接表的连接字段创建UNIQUE的约束，这样才能是一对一，否则是一对多
    fk_student_id = Column(
        Integer,
        ForeignKey(
            "studentsNumberInfo.id",
            ondelete="CASCADE",
            onupdate="CASCADE"
        ),
        nullable=False,
        comment="学生编号"
    )
    # 相比于一对一，连接表的连接字段不用UNIQUE约束即为多对一关系
    fk_class_id = Column(
        Integer,
        ForeignKey(
            "classesInfo.id",
            ondelete="CASCADE",
            onupdate="CASCADE"
        ),
        comment="班级编号"
    )
    # 逻辑字段
    # 所在班级，这里是一对多关系，一个班级中可以有多名学生
    from_class = relationship(
        "ClassesInfo",
        backref="have_student",
    )
    # 学生学号，这里是一对一关系，一个学生只能拥有一个学号
    number_info = relationship(
        "StudentsNumberInfo",
        backref="student_info",
    )
if __name__ == "__main__":
    # 删除表
    Base.metadata.drop_all(engine)
    # 创建表
    Base.metadata.create_all(engine)
```

```
2026-05-07 13:11:37,198 INFO sqlalchemy.engine.Engine select pg_catalog.version()
2026-05-07 13:11:37,199 INFO sqlalchemy.engine.Engine [raw sql] {}
2026-05-07 13:11:37,214 INFO sqlalchemy.engine.Engine select current_schema()
2026-05-07 13:11:37,217 INFO sqlalchemy.engine.Engine [raw sql] {}
2026-05-07 13:11:37,235 INFO sqlalchemy.engine.Engine show standard_conforming_strings
2026-05-07 13:11:37,236 INFO sqlalchemy.engine.Engine [raw sql] {}
2026-05-07 13:11:37,252 INFO sqlalchemy.engine.Engine BEGIN (implicit)
2026-05-07 13:11:37,256 INFO sqlalchemy.engine.Engine SELECT pg_catalog.pg_class.relname
FROM pg_catalog.pg_class JOIN pg_catalog.pg_namespace ON pg_catalog.pg_namespace.oid = pg_cata
log.pg_class.relnamespace
WHERE pg_catalog.pg_class.relname = %(table_name)s AND pg_catalog.pg_class.relkind = ANY (ARRA
Y[%(param_1)s, %(param_2)s, %(param_3)s, %(param_4)s, %(param_5)s]) AND pg_catalog.pg_table_is
_visible(pg_catalog.pg_class.oid) AND pg_catalog.pg_namespace.nspname != %(nspname_1)s
```

```
C:\Users\gzs27\AppData\Local\Temp\ipykernel_4476\3227781301.py:4: MovedIn20Warning: The ``decl
arative_base()`` function is now available as sqlalchemy.orm.declarative_base(). (deprecated s
ince: 2.0) (Background on SQLAlchemy 2.0 at: https://sqlalche.me/e/b8d9)
  Base = declarative_base()
```

```
2026-05-07 13:11:37,256 INFO sqlalchemy.engine.Engine [generated in 0.00087s] {'table_name':
'studentsNumberInfo', 'param_1': 'r', 'param_2': 'p', 'param_3': 'f', 'param_4': 'v', 'param_
5': 'm', 'nspname_1': 'pg_catalog'}
2026-05-07 13:11:37,279 INFO sqlalchemy.engine.Engine SELECT pg_catalog.pg_class.relname
FROM pg_catalog.pg_class JOIN pg_catalog.pg_namespace ON pg_catalog.pg_namespace.oid = pg_cata
log.pg_class.relnamespace
WHERE pg_catalog.pg_class.relname = %(table_name)s AND pg_catalog.pg_class.relkind = ANY (ARRA
Y[%(param_1)s, %(param_2)s, %(param_3)s, %(param_4)s, %(param_5)s]) AND pg_catalog.pg_table_is
_visible(pg_catalog.pg_class.oid) AND pg_catalog.pg_namespace.nspname != %(nspname_1)s
2026-05-07 13:11:37,280 INFO sqlalchemy.engine.Engine [cached since 0.02391s ago] {'table_nam
e': 'teachersInfo', 'param_1': 'r', 'param_2': 'p', 'param_3': 'f', 'param_4': 'v', 'param_5':
'm', 'nspname_1': 'pg_catalog'}
2026-05-07 13:11:37,289 INFO sqlalchemy.engine.Engine SELECT pg_catalog.pg_class.relname
FROM pg_catalog.pg_class JOIN pg_catalog.pg_namespace ON pg_catalog.pg_namespace.oid = pg_cata
log.pg_class.relnamespace
WHERE pg_catalog.pg_class.relname = %(table_name)s AND pg_catalog.pg_class.relkind = ANY (ARRA
Y[%(param_1)s, %(param_2)s, %(param_3)s, %(param_4)s, %(param_5)s]) AND pg_catalog.pg_table_is
_visible(pg_catalog.pg_class.oid) AND pg_catalog.pg_namespace.nspname != %(nspname_1)s
2026-05-07 13:11:37,290 INFO sqlalchemy.engine.Engine [cached since 0.03374s ago] {'table_nam
e': 'classesInfo', 'param_1': 'r', 'param_2': 'p', 'param_3': 'f', 'param_4': 'v', 'param_5':
'm', 'nspname_1': 'pg_catalog'}
2026-05-07 13:11:37,304 INFO sqlalchemy.engine.Engine SELECT pg_catalog.pg_class.relname
FROM pg_catalog.pg_class JOIN pg_catalog.pg_namespace ON pg_catalog.pg_namespace.oid = pg_cata
log.pg_class.relnamespace
WHERE pg_catalog.pg_class.relname = %(table_name)s AND pg_catalog.pg_class.relkind = ANY (ARRA
Y[%(param_1)s, %(param_2)s, %(param_3)s, %(param_4)s, %(param_5)s]) AND pg_catalog.pg_table_is
_visible(pg_catalog.pg_class.oid) AND pg_catalog.pg_namespace.nspname != %(nspname_1)s
2026-05-07 13:11:37,306 INFO sqlalchemy.engine.Engine [cached since 0.04949s ago] {'table_nam
e': 'classesAndTeachersRelationship', 'param_1': 'r', 'param_2': 'p', 'param_3': 'f', 'param_
4': 'v', 'param_5': 'm', 'nspname_1': 'pg_catalog'}
2026-05-07 13:11:37,316 INFO sqlalchemy.engine.Engine SELECT pg_catalog.pg_class.relname
FROM pg_catalog.pg_class JOIN pg_catalog.pg_namespace ON pg_catalog.pg_namespace.oid = pg_cata
log.pg_class.relnamespace
WHERE pg_catalog.pg_class.relname = %(table_name)s AND pg_catalog.pg_class.relkind = ANY (ARRA
Y[%(param_1)s, %(param_2)s, %(param_3)s, %(param_4)s, %(param_5)s]) AND pg_catalog.pg_table_is
_visible(pg_catalog.pg_class.oid) AND pg_catalog.pg_namespace.nspname != %(nspname_1)s
2026-05-07 13:11:37,317 INFO sqlalchemy.engine.Engine [cached since 0.06137s ago] {'table_nam
e': 'studentsInfo', 'param_1': 'r', 'param_2': 'p', 'param_3': 'f', 'param_4': 'v', 'param_5':
'm', 'nspname_1': 'pg_catalog'}
2026-05-07 13:11:37,326 INFO sqlalchemy.engine.Engine COMMIT
2026-05-07 13:11:37,334 INFO sqlalchemy.engine.Engine BEGIN (implicit)
2026-05-07 13:11:37,335 INFO sqlalchemy.engine.Engine SELECT pg_catalog.pg_class.relname
FROM pg_catalog.pg_class JOIN pg_catalog.pg_namespace ON pg_catalog.pg_namespace.oid = pg_cata
log.pg_class.relnamespace
WHERE pg_catalog.pg_class.relname = %(table_name)s AND pg_catalog.pg_class.relkind = ANY (ARRA
Y[%(param_1)s, %(param_2)s, %(param_3)s, %(param_4)s, %(param_5)s]) AND pg_catalog.pg_table_is
_visible(pg_catalog.pg_class.oid) AND pg_catalog.pg_namespace.nspname != %(nspname_1)s
2026-05-07 13:11:37,336 INFO sqlalchemy.engine.Engine [cached since 0.08027s ago] {'table_nam
e': 'studentsNumberInfo', 'param_1': 'r', 'param_2': 'p', 'param_3': 'f', 'param_4': 'v', 'par
am_5': 'm', 'nspname_1': 'pg_catalog'}
2026-05-07 13:11:37,352 INFO sqlalchemy.engine.Engine SELECT pg_catalog.pg_class.relname
FROM pg_catalog.pg_class JOIN pg_catalog.pg_namespace ON pg_catalog.pg_namespace.oid = pg_cata
log.pg_class.relnamespace
WHERE pg_catalog.pg_class.relname = %(table_name)s AND pg_catalog.pg_class.relkind = ANY (ARRA
Y[%(param_1)s, %(param_2)s, %(param_3)s, %(param_4)s, %(param_5)s]) AND pg_catalog.pg_table_is
_visible(pg_catalog.pg_class.oid) AND pg_catalog.pg_namespace.nspname != %(nspname_1)s
2026-05-07 13:11:37,353 INFO sqlalchemy.engine.Engine [cached since 0.09702s ago] {'table_nam
e': 'teachersInfo', 'param_1': 'r', 'param_2': 'p', 'param_3': 'f', 'param_4': 'v', 'param_5':
'm', 'nspname_1': 'pg_catalog'}
2026-05-07 13:11:37,361 INFO sqlalchemy.engine.Engine SELECT pg_catalog.pg_class.relname
FROM pg_catalog.pg_class JOIN pg_catalog.pg_namespace ON pg_catalog.pg_namespace.oid = pg_cata
log.pg_class.relnamespace
WHERE pg_catalog.pg_class.relname = %(table_name)s AND pg_catalog.pg_class.relkind = ANY (ARRA
Y[%(param_1)s, %(param_2)s, %(param_3)s, %(param_4)s, %(param_5)s]) AND pg_catalog.pg_table_is
_visible(pg_catalog.pg_class.oid) AND pg_catalog.pg_namespace.nspname != %(nspname_1)s
2026-05-07 13:11:37,362 INFO sqlalchemy.engine.Engine [cached since 0.1064s ago] {'table_nam
```

```
e': 'classesInfo', 'param_1': 'r', 'param_2': 'p', 'param_3': 'f', 'param_4': 'v', 'param_5':
'm', 'nspname_1': 'pg_catalog'}
2026-05-07 13:11:37,372 INFO sqlalchemy.engine.Engine SELECT pg_catalog.pg_class.relname
FROM pg_catalog.pg_class JOIN pg_catalog.pg_namespace ON pg_catalog.pg_namespace.oid = pg_cata
log.pg_class.relnamespace
WHERE pg_catalog.pg_class.relname = %(table_name)s AND pg_catalog.pg_class.relkind = ANY (ARRA
Y[%(param_1)s, %(param_2)s, %(param_3)s, %(param_4)s, %(param_5)s]) AND pg_catalog.pg_table_is
_visible(pg_catalog.pg_class.oid) AND pg_catalog.pg_namespace.nspname != %(nspname_1)s
2026-05-07 13:11:37,372 INFO sqlalchemy.engine.Engine [cached since 0.117s ago] {'table_name':
'classesAndTeachersRelationship', 'param_1': 'r', 'param_2': 'p', 'param_3': 'f', 'param_4':
'v', 'param_5': 'm', 'nspname_1': 'pg_catalog'}
2026-05-07 13:11:37,382 INFO sqlalchemy.engine.Engine SELECT pg_catalog.pg_class.relname
FROM pg_catalog.pg_class JOIN pg_catalog.pg_namespace ON pg_catalog.pg_namespace.oid = pg_cata
log.pg_class.relnamespace
WHERE pg_catalog.pg_class.relname = %(table_name)s AND pg_catalog.pg_class.relkind = ANY (ARRA
Y[%(param_1)s, %(param_2)s, %(param_3)s, %(param_4)s, %(param_5)s]) AND pg_catalog.pg_table_is
_visible(pg_catalog.pg_class.oid) AND pg_catalog.pg_namespace.nspname != %(nspname_1)s
2026-05-07 13:11:37,382 INFO sqlalchemy.engine.Engine [cached since 0.1261s ago] {'table_nam
e': 'studentsInfo', 'param_1': 'r', 'param_2': 'p', 'param_3': 'f', 'param_4': 'v', 'param_5':
'm', 'nspname_1': 'pg_catalog'}
2026-05-07 13:11:37,391 INFO sqlalchemy.engine.Engine
CREATE TABLE "studentsNumberInfo" (
        id SERIAL NOT NULL,
        number INTEGER NOT NULL,
        admission DATE NOT NULL,
        graduation DATE NOT NULL,
        PRIMARY KEY (id),
        UNIQUE (number)
)


2026-05-07 13:11:37,392 INFO sqlalchemy.engine.Engine [no key 0.00127s] {}
2026-05-07 13:11:37,423 INFO sqlalchemy.engine.Engine COMMENT ON COLUMN "studentsNumberInfo".i
d IS '主键'
2026-05-07 13:11:37,424 INFO sqlalchemy.engine.Engine [no key 0.00094s] {}
2026-05-07 13:11:37,435 INFO sqlalchemy.engine.Engine COMMENT ON COLUMN "studentsNumberInfo".n
umber IS '学生编号'
2026-05-07 13:11:37,436 INFO sqlalchemy.engine.Engine [no key 0.00074s] {}
2026-05-07 13:11:37,444 INFO sqlalchemy.engine.Engine COMMENT ON COLUMN "studentsNumberInfo".a
dmission IS '入学时间'
2026-05-07 13:11:37,444 INFO sqlalchemy.engine.Engine [no key 0.00074s] {}
2026-05-07 13:11:37,452 INFO sqlalchemy.engine.Engine COMMENT ON COLUMN "studentsNumberInfo".g
raduation IS '毕业时间'
2026-05-07 13:11:37,454 INFO sqlalchemy.engine.Engine [no key 0.00101s] {}
2026-05-07 13:11:37,463 INFO sqlalchemy.engine.Engine
CREATE TABLE "teachersInfo" (
        id SERIAL NOT NULL,
        number INTEGER NOT NULL,
        name VARCHAR(64) NOT NULL,
        gender VARCHAR(1) NOT NULL,
        age INTEGER NOT NULL,
        PRIMARY KEY (id),
        UNIQUE (number)
)


2026-05-07 13:11:37,463 INFO sqlalchemy.engine.Engine [no key 0.00064s] {}
2026-05-07 13:11:37,472 INFO sqlalchemy.engine.Engine COMMENT ON COLUMN "teachersInfo".id IS
'主键'
2026-05-07 13:11:37,473 INFO sqlalchemy.engine.Engine [no key 0.00068s] {}
2026-05-07 13:11:37,483 INFO sqlalchemy.engine.Engine COMMENT ON COLUMN "teachersInfo".number
IS '教师编号'
2026-05-07 13:11:37,484 INFO sqlalchemy.engine.Engine [no key 0.00072s] {}
2026-05-07 13:11:37,492 INFO sqlalchemy.engine.Engine COMMENT ON COLUMN "teachersInfo".name IS
'教师姓名'
2026-05-07 13:11:37,493 INFO sqlalchemy.engine.Engine [no key 0.00090s] {}
```

```
2026-05-07 13:11:37,502 INFO sqlalchemy.engine.Engine COMMENT ON COLUMN "teachersInfo".gender
IS '教师性别'
2026-05-07 13:11:37,502 INFO sqlalchemy.engine.Engine [no key 0.00100s] {}
2026-05-07 13:11:37,510 INFO sqlalchemy.engine.Engine COMMENT ON COLUMN "teachersInfo".age IS
'教师年龄'
2026-05-07 13:11:37,512 INFO sqlalchemy.engine.Engine [no key 0.00075s] {}
2026-05-07 13:11:37,522 INFO sqlalchemy.engine.Engine
CREATE TABLE "classesInfo" (
        id SERIAL NOT NULL,
        number INTEGER NOT NULL,
        name VARCHAR(64) NOT NULL,
        fk_teacher_id INTEGER NOT NULL,
        PRIMARY KEY (id),
        UNIQUE (number),
        UNIQUE (name),
        UNIQUE (fk_teacher_id),
        FOREIGN KEY(fk_teacher_id) REFERENCES "teachersInfo" (id) ON DELETE CASCADE ON UPDATE
CASCADE
)


2026-05-07 13:11:37,523 INFO sqlalchemy.engine.Engine [no key 0.00073s] {}
2026-05-07 13:11:37,540 INFO sqlalchemy.engine.Engine COMMENT ON COLUMN "classesInfo".id IS
'主键'
2026-05-07 13:11:37,540 INFO sqlalchemy.engine.Engine [no key 0.00068s] {}
2026-05-07 13:11:37,552 INFO sqlalchemy.engine.Engine COMMENT ON COLUMN "classesInfo".number I
S '班级编号'
2026-05-07 13:11:37,553 INFO sqlalchemy.engine.Engine [no key 0.00083s] {}
2026-05-07 13:11:37,561 INFO sqlalchemy.engine.Engine COMMENT ON COLUMN "classesInfo".name IS
'班级名称'
2026-05-07 13:11:37,562 INFO sqlalchemy.engine.Engine [no key 0.00057s] {}
2026-05-07 13:11:37,570 INFO sqlalchemy.engine.Engine COMMENT ON COLUMN "classesInfo".fk_teach
er_id IS '班级负责人'
2026-05-07 13:11:37,570 INFO sqlalchemy.engine.Engine [no key 0.00072s] {}
2026-05-07 13:11:37,580 INFO sqlalchemy.engine.Engine
CREATE TABLE "classesAndTeachersRelationship" (
        id SERIAL NOT NULL,
        fk_teacher_id INTEGER NOT NULL,
        fk_class_id INTEGER NOT NULL,
        PRIMARY KEY (id),
        UNIQUE (fk_teacher_id, fk_class_id),
        FOREIGN KEY(fk_teacher_id) REFERENCES "teachersInfo" (id) ON DELETE CASCADE ON UPDATE
CASCADE,
        FOREIGN KEY(fk_class_id) REFERENCES "classesInfo" (id) ON DELETE CASCADE ON UPDATE CAS
CADE
)


2026-05-07 13:11:37,580 INFO sqlalchemy.engine.Engine [no key 0.00076s] {}
2026-05-07 13:11:37,591 INFO sqlalchemy.engine.Engine COMMENT ON COLUMN "classesAndTeachersRel
ationship".id IS '主键'
2026-05-07 13:11:37,592 INFO sqlalchemy.engine.Engine [no key 0.00085s] {}
2026-05-07 13:11:37,599 INFO sqlalchemy.engine.Engine COMMENT ON COLUMN "classesAndTeachersRel
ationship".fk_teacher_id IS '教师记录'
2026-05-07 13:11:37,599 INFO sqlalchemy.engine.Engine [no key 0.00060s] {}
2026-05-07 13:11:37,608 INFO sqlalchemy.engine.Engine COMMENT ON COLUMN "classesAndTeachersRel
ationship".fk_class_id IS '班级记录'
2026-05-07 13:11:37,609 INFO sqlalchemy.engine.Engine [no key 0.00085s] {}
2026-05-07 13:11:37,618 INFO sqlalchemy.engine.Engine
CREATE TABLE "studentsInfo" (
        id SERIAL NOT NULL,
        name VARCHAR(64) NOT NULL,
        gender VARCHAR(1) NOT NULL,
        age INTEGER NOT NULL,
        fk_student_id INTEGER NOT NULL,
        fk_class_id INTEGER,
```

```
        PRIMARY KEY (id),
        FOREIGN KEY(fk_student_id) REFERENCES "studentsNumberInfo" (id) ON DELETE CASCADE ON U
PDATE CASCADE,
        FOREIGN KEY(fk_class_id) REFERENCES "classesInfo" (id) ON DELETE CASCADE ON UPDATE CAS
CADE
)


2026-05-07 13:11:37,619 INFO sqlalchemy.engine.Engine [no key 0.00131s] {}
2026-05-07 13:11:37,630 INFO sqlalchemy.engine.Engine COMMENT ON COLUMN "studentsInfo".id IS
'主键'
2026-05-07 13:11:37,632 INFO sqlalchemy.engine.Engine [no key 0.00089s] {}
2026-05-07 13:11:37,639 INFO sqlalchemy.engine.Engine COMMENT ON COLUMN "studentsInfo".name IS
'学生姓名'
2026-05-07 13:11:37,639 INFO sqlalchemy.engine.Engine [no key 0.00075s] {}
2026-05-07 13:11:37,647 INFO sqlalchemy.engine.Engine COMMENT ON COLUMN "studentsInfo".gender
IS '学生性别'
2026-05-07 13:11:37,648 INFO sqlalchemy.engine.Engine [no key 0.00111s] {}
2026-05-07 13:11:37,658 INFO sqlalchemy.engine.Engine COMMENT ON COLUMN "studentsInfo".age IS
'学生年龄'
2026-05-07 13:11:37,658 INFO sqlalchemy.engine.Engine [no key 0.00071s] {}
2026-05-07 13:11:37,666 INFO sqlalchemy.engine.Engine COMMENT ON COLUMN "studentsInfo".fk_stud
ent_id IS '学生编号'
2026-05-07 13:11:37,667 INFO sqlalchemy.engine.Engine [no key 0.00108s] {}
2026-05-07 13:11:37,675 INFO sqlalchemy.engine.Engine COMMENT ON COLUMN "studentsInfo".fk_clas
s_id IS '班级编号'
2026-05-07 13:11:37,676 INFO sqlalchemy.engine.Engine [no key 0.00081s] {}
2026-05-07 13:11:37,684 INFO sqlalchemy.engine.Engine COMMIT
```

插入数据

In [5]:

```python
import datetime
session.add_all(
    (
        # 插入学号表数据
        StudentsNumberInfo(
            number=160201,
            admission=datetime.datetime.date(datetime.datetime(2016, 9, 1)),
            graduation=datetime.datetime.date(datetime.datetime(2021, 6, 15))
        ),
        StudentsNumberInfo(
            number=160101,
            admission=datetime.datetime.date(datetime.datetime(2016, 9, 1)),
            graduation=datetime.datetime.date(datetime.datetime(2021, 6, 15))
        ),
        StudentsNumberInfo(
            number=160301,
            admission=datetime.datetime.date(datetime.datetime(2016, 9, 1)),
            graduation=datetime.datetime.date(datetime.datetime(2021, 6, 15))
        ),
        StudentsNumberInfo(
            number=160102,
            admission=datetime.datetime.date(datetime.datetime(2016, 9, 1)),
            graduation=datetime.datetime.date(datetime.datetime(2021, 6, 15))
        ),
        StudentsNumberInfo(
            number=160302,
            admission=datetime.datetime.date(datetime.datetime(2016, 9, 1)),
            graduation=datetime.datetime.date(datetime.datetime(2021, 6, 15))
        ),
        StudentsNumberInfo(
            number=160202,
            admission=datetime.datetime.date(datetime.datetime(2016, 9, 1)),
            graduation=datetime.datetime.date(datetime.datetime(2021, 6, 15))
        ),
```

```
        # 插入教师表数据
        TeachersInfo(
            number=3341, name="David", gender="m", age=32,
        ),
        TeachersInfo(
            number=3342, name="Jason", gender="m", age=30,
        ),
        TeachersInfo(
            number=3343, name="Lisa", gender="f", age=28,
        ),
        # 插入班级表数据
        ClassesInfo(
            number=1601, name="one year one class", fk_teacher_id=1
        ),
        ClassesInfo(
            number=1602, name="one year two class", fk_teacher_id=2
        ),
        ClassesInfo(
            number=1603, name="one year three class", fk_teacher_id=3
        ),
        # 插入中间表数据
        ClassesAndTeachersRelationship(
            fk_class_id=1, fk_teacher_id=1
        ),
        ClassesAndTeachersRelationship(
            fk_class_id=2, fk_teacher_id=1
        ),
        ClassesAndTeachersRelationship(
            fk_class_id=3, fk_teacher_id=1
        ),
        ClassesAndTeachersRelationship(
            fk_class_id=1, fk_teacher_id=2
        ),
        ClassesAndTeachersRelationship(
            fk_class_id=3, fk_teacher_id=3
        ),
        # 插入学生表数据
        StudentsInfo(
            name="Jack", gender="m", age=17, fk_student_id=1, fk_class_id=2
        ),
        StudentsInfo(
            name="Tom", gender="m", age=18, fk_student_id=2, fk_class_id=1
        ),
        StudentsInfo(
            name="Mary", gender="f", age=16, fk_student_id=3,
            fk_class_id=3
        ),
        StudentsInfo(
            name="Anna", gender="f", age=17, fk_student_id=4,
            fk_class_id=1
        ),
        StudentsInfo(
            name="Bobby", gender="m", age=18, fk_student_id=6, fk_class_id=2
        ),
    )
)
session.commit()
# 关闭链接，亦可使用session.remove()，它将回收该链接
session.close()
```

```
2026-05-07 13:14:43,501 INFO sqlalchemy.engine.Engine BEGIN (implicit)
2026-05-07 13:14:43,504 INFO sqlalchemy.engine.Engine INSERT INTO "studentsNumberInfo" (numbe
r, admission, graduation) SELECT p0::INTEGER, p1::DATE, p2::DATE FROM (VALUES (%(number__0)s,
%(admission__0)s, %(graduation__0)s, 0), (%(number__1)s, %(admission__1)s, %(graduation__1)s,
1), (%(number__2)s, %(a ... 233 characters truncated ... en_counter) ORDER BY sen_counter RETU
RNING "studentsNumberInfo".id, "studentsNumberInfo".id AS id__1
2026-05-07 13:14:43,505 INFO sqlalchemy.engine.Engine [generated in 0.00016s (insertmanyvalue
s) 1/1 (ordered)] {'number__0': 160201, 'admission__0': datetime.date(2016, 9, 1), 'graduation
__0': datetime.date(2021, 6, 15), 'number__1': 160101, 'admission__1': datetime.date(2016, 9,
1), 'graduation__1': datetime.date(2021, 6, 15), 'number__2': 160301, 'admission__2': datetim
e.date(2016, 9, 1), 'graduation__2': datetime.date(2021, 6, 15), 'number__3': 160102, 'admissi
on__3': datetime.date(2016, 9, 1), 'graduation__3': datetime.date(2021, 6, 15), 'number__4': 1
60302, 'admission__4': datetime.date(2016, 9, 1), 'graduation__4': datetime.date(2021, 6, 15),
'number__5': 160202, 'admission__5': datetime.date(2016, 9, 1), 'graduation__5': datetime.date
(2021, 6, 15)}
2026-05-07 13:14:43,532 INFO sqlalchemy.engine.Engine INSERT INTO "teachersInfo" (number, nam
e, gender, age) SELECT p0::INTEGER, p1::VARCHAR, p2::VARCHAR, p3::INTEGER FROM (VALUES (%(numb
er__0)s, %(name__0)s, %(gender__0)s, %(age__0)s, 0), (%(number__1)s, %(name__1)s, %(gender__1)
s, %(age__1)s, 1), (%(number__2)s, %(name__2)s, %(gender__2)s, %(age__2)s, 2)) AS imp_sen(p0,
p1, p2, p3, sen_counter) ORDER BY sen_counter RETURNING "teachersInfo".id, "teachersInfo".id A
S id__1
2026-05-07 13:14:43,532 INFO sqlalchemy.engine.Engine [generated in 0.00009s (insertmanyvalue
s) 1/1 (ordered)] {'number__0': 3341, 'age__0': 32, 'gender__0': 'm', 'name__0': 'David', 'num
ber__1': 3342, 'age__1': 30, 'gender__1': 'm', 'name__1': 'Jason', 'number__2': 3343, 'age__
2': 28, 'gender__2': 'f', 'name__2': 'Lisa'}
2026-05-07 13:14:43,543 INFO sqlalchemy.engine.Engine INSERT INTO "classesInfo" (number, name,
fk_teacher_id) SELECT p0::INTEGER, p1::VARCHAR, p2::INTEGER FROM (VALUES (%(number__0)s, %(nam
e__0)s, %(fk_teacher_id__0)s, 0), (%(number__1)s, %(name__1)s, %(fk_teacher_id__1)s, 1), (%(nu
mber__2)s, %(name__2)s, %(fk_teacher_id__2)s, 2)) AS imp_sen(p0, p1, p2, sen_counter) ORDER BY
sen_counter RETURNING "classesInfo".id, "classesInfo".id AS id__1
2026-05-07 13:14:43,544 INFO sqlalchemy.engine.Engine [generated in 0.00007s (insertmanyvalue
s) 1/1 (ordered)] {'number__0': 1601, 'fk_teacher_id__0': 1, 'name__0': 'one year one class',
'number__1': 1602, 'fk_teacher_id__1': 2, 'name__1': 'one year two class', 'number__2': 1603,
'fk_teacher_id__2': 3, 'name__2': 'one year three class'}
2026-05-07 13:14:43,559 INFO sqlalchemy.engine.Engine INSERT INTO "classesAndTeachersRelations
hip" (fk_teacher_id, fk_class_id) SELECT p0::INTEGER, p1::INTEGER FROM (VALUES (%(fk_teacher_i
d__0)s, %(fk_class_id__0)s, 0), (%(fk_teacher_id__1)s, %(fk_class_id__1)s, 1), (%(fk_teacher_i
d__2)s, %(fk_class_id__2)s, 2), (%(fk_teacher_id__3)s, %(fk_class_id__3)s, 3), (%(fk_teacher_i
d__4)s, %(fk_class_id__4)s, 4)) AS imp_sen(p0, p1, sen_counter) ORDER BY sen_counter RETURNING
"classesAndTeachersRelationship".id, "classesAndTeachersRelationship".id AS id__1
2026-05-07 13:14:43,560 INFO sqlalchemy.engine.Engine [generated in 0.00009s (insertmanyvalue
s) 1/1 (ordered)] {'fk_teacher_id__0': 1, 'fk_class_id__0': 1, 'fk_teacher_id__1': 1, 'fk_clas
s_id__1': 2, 'fk_teacher_id__2': 1, 'fk_class_id__2': 3, 'fk_teacher_id__3': 2, 'fk_class_id__
3': 1, 'fk_teacher_id__4': 3, 'fk_class_id__4': 3}
2026-05-07 13:14:43,572 INFO sqlalchemy.engine.Engine INSERT INTO "studentsInfo" (name, gende
r, age, fk_student_id, fk_class_id) SELECT p0::VARCHAR, p1::VARCHAR, p2::INTEGER, p3::INTEGER,
p4::INTEGER FROM (VALUES (%(name__0)s, %(gender__0)s, %(age__0)s, %(fk_student_id__0)s, %(fk_c
lass_id__0)s, 0), (%(n ... 364 characters truncated ... 2, p3, p4, sen_counter) ORDER BY sen_c
ounter RETURNING "studentsInfo".id, "studentsInfo".id AS id__1
2026-05-07 13:14:43,573 INFO sqlalchemy.engine.Engine [generated in 0.00009s (insertmanyvalue
s) 1/1 (ordered)] {'age__0': 17, 'gender__0': 'm', 'fk_student_id__0': 1, 'fk_class_id__0': 2,
'name__0': 'Jack', 'age__1': 18, 'gender__1': 'm', 'fk_student_id__1': 2, 'fk_class_id__1': 1,
'name__1': 'Tom', 'age__2': 16, 'gender__2': 'f', 'fk_student_id__2': 3, 'fk_class_id__2': 3,
'name__2': 'Mary', 'age__3': 17, 'gender__3': 'f', 'fk_student_id__3': 4, 'fk_class_id__3': 1,
'name__3': 'Anna', 'age__4': 18, 'gender__4': 'm', 'fk_student_id__4': 6, 'fk_class_id__4': 2,
'name__4': 'Bobby'}
2026-05-07 13:14:43,583 INFO sqlalchemy.engine.Engine COMMIT
```

JOIN

In [6]:

```
Session = sessionmaker(bind=engine)
session = scoped_session(Session)
result = session.query(
    StudentsInfo.name,
```

```
    StudentsNumberInfo.number,
    ClassesInfo.number
).join(
    StudentsNumberInfo,
    StudentsInfo.fk_student_id == StudentsNumberInfo.id
).join(
    ClassesInfo,
    StudentsInfo.fk_class_id == ClassesInfo.id
).all()
print(result)
# [('Jack', 160201, 1602), ('Tom', 160101, 1601), ('Mary', 160301, 1603), ('Anna', 160102, 16
# 关闭链接，亦可使用session.remove()，它将回收该链接
session.close()
```

```
2026-05-07 13:14:46,944 INFO sqlalchemy.engine.Engine BEGIN (implicit)
2026-05-07 13:14:46,947 INFO sqlalchemy.engine.Engine SELECT "studentsInfo".name AS "studentsI
nfo_name", "studentsNumberInfo".number AS "studentsNumberInfo_number", "classesInfo".number AS
"classesInfo_number"
FROM "studentsInfo" JOIN "studentsNumberInfo" ON "studentsInfo".fk_student_id = "studentsNumbe
rInfo".id JOIN "classesInfo" ON "studentsInfo".fk_class_id = "classesInfo".id
2026-05-07 13:14:46,948 INFO sqlalchemy.engine.Engine [generated in 0.00120s] {}
[('Jack', 160201, 1602), ('Tom', 160101, 1601), ('Mary', 160301, 1603), ('Anna', 160102, 160
1), ('Bobby', 160202, 1602)]
2026-05-07 13:14:46,970 INFO sqlalchemy.engine.Engine ROLLBACK
```

LEFT JOIN

left join只需要在每个JOIN中指定isouter关键字参数为True即可：

session.query(左表.字段, 右表.字段) .join(右表, 链接条件, isouter=True).all()

RIGHT JOIN

需要换表的位置，SQLALchemy本身并未提供RIGHT JOIN，所以使用时一定要注意驱动顺序，小表驱动大表：

session.query(左表.字段, 右表.字段) .join(左表, 链接条件, isouter=True).all()

子查询

子查询使用subquery()实现，如下所示，查询每个班级中年龄最小的人：

In [7]:
```
# 获取链接池、ORM表对象
from sqlalchemy import func
# 子查询中所有字段的访问都需要加上c的前缀
# 如 sub_query.c.id、 sub_query.c.name等
sub_query = session.query(
    # 使用label()来为字段AS一个别名
    # 后续访问需要通过sub_query.c.alias进行访问
    func.min(StudentsInfo.age).label("min_age"),
    ClassesInfo.id,
    ClassesInfo.name
).join(
    ClassesInfo,
    StudentsInfo.fk_class_id == ClassesInfo.id
).group_by(
    ClassesInfo.id
).subquery()

result = session.query(
    StudentsInfo.name,
```

```
    sub_query.c.min_age,
    sub_query.c.name
).join(
    sub_query,
    sub_query.c.id == StudentsInfo.fk_class_id
).filter(
   sub_query.c.min_age == StudentsInfo.age
)

print(result.all())
# [('Jack', 17, 'one year two class'), ('Mary', 16, 'one year three class'), ('Anna', 17, 'one

# 关闭链接，亦可使用session.remove()，它将回收该链接
session.close()
```

```
2026-05-07 13:14:50,299 INFO sqlalchemy.engine.Engine BEGIN (implicit)
2026-05-07 13:14:50,302 INFO sqlalchemy.engine.Engine SELECT "studentsInfo".name AS "studentsI
nfo_name", anon_1.min_age AS anon_1_min_age, anon_1.name AS anon_1_name
FROM "studentsInfo" JOIN (SELECT min("studentsInfo".age) AS min_age, "classesInfo".id AS id,
"classesInfo".name AS name
FROM "studentsInfo" JOIN "classesInfo" ON "studentsInfo".fk_class_id = "classesInfo".id GROUP
BY "classesInfo".id) AS anon_1 ON anon_1.id = "studentsInfo".fk_class_id
WHERE anon_1.min_age = "studentsInfo".age
2026-05-07 13:14:50,302 INFO sqlalchemy.engine.Engine [generated in 0.00103s] {}
[('Jack', 17, 'one year two class'), ('Mary', 16, 'one year three class'), ('Anna', 17, 'one y
ear one class')]
2026-05-07 13:14:50,322 INFO sqlalchemy.engine.Engine ROLLBACK
```

正反查询

上面我们都是通过JOIN进行查询的，实际上我们也可以通过逻辑字段relationship进行查询。

正向查询

下面是正向查询的示例，正向查询是指从有relationship逻辑字段的表开始查询：

In [8]:
```
# 查询所有学生的所在班级，我们可以通过学生的from_class字段拿到其所在班级
# 另外，对于学生来说，班级只能有一个，所以have_student应当是一个对象
# 获取链接池、ORM表对象
students_lst = session.query(
    StudentsInfo
).all()
for row in students_lst:
    print(f"""
             student name : {row.name}
             from : {row.from_class.name}
          """)
# student name : Mary
# from : one year three class
# student name : Anna
# from : one year one class
# student name : Bobby
# from : one year two class
# 关闭链接，亦可使用session.remove()，它将回收该链接
session.close()
```

```
2026-05-07 13:14:59,992 INFO sqlalchemy.engine.Engine BEGIN (implicit)
2026-05-07 13:14:59,995 INFO sqlalchemy.engine.Engine SELECT "studentsInfo".id AS "studentsInf
o_id", "studentsInfo".name AS "studentsInfo_name", "studentsInfo".gender AS "studentsInfo_gend
er", "studentsInfo".age AS "studentsInfo_age", "studentsInfo".fk_student_id AS "studentsInfo_f
k_student_id", "studentsInfo".fk_class_id AS "studentsInfo_fk_class_id"
FROM "studentsInfo"
2026-05-07 13:14:59,995 INFO sqlalchemy.engine.Engine [generated in 0.00129s] {}
2026-05-07 13:15:00,013 INFO sqlalchemy.engine.Engine SELECT "classesInfo".id AS "classesInfo_
id", "classesInfo".number AS "classesInfo_number", "classesInfo".name AS "classesInfo_name",
"classesInfo".fk_teacher_id AS "classesInfo_fk_teacher_id"
FROM "classesInfo"
WHERE "classesInfo".id = %(pk_1)s
2026-05-07 13:15:00,015 INFO sqlalchemy.engine.Engine [generated in 0.00144s] {'pk_1': 2}

            student name : Jack
            from : one year two class

2026-05-07 13:15:00,026 INFO sqlalchemy.engine.Engine SELECT "classesInfo".id AS "classesInfo_
id", "classesInfo".number AS "classesInfo_number", "classesInfo".name AS "classesInfo_name",
"classesInfo".fk_teacher_id AS "classesInfo_fk_teacher_id"
FROM "classesInfo"
WHERE "classesInfo".id = %(pk_1)s
2026-05-07 13:15:00,027 INFO sqlalchemy.engine.Engine [cached since 0.01387s ago] {'pk_1': 1}

            student name : Tom
            from : one year one class

2026-05-07 13:15:00,056 INFO sqlalchemy.engine.Engine SELECT "classesInfo".id AS "classesInfo_
id", "classesInfo".number AS "classesInfo_number", "classesInfo".name AS "classesInfo_name",
"classesInfo".fk_teacher_id AS "classesInfo_fk_teacher_id"
FROM "classesInfo"
WHERE "classesInfo".id = %(pk_1)s
2026-05-07 13:15:00,057 INFO sqlalchemy.engine.Engine [cached since 0.04316s ago] {'pk_1': 3}

            student name : Mary
            from : one year three class


            student name : Anna
            from : one year one class


            student name : Bobby
            from : one year two class

2026-05-07 13:15:00,064 INFO sqlalchemy.engine.Engine ROLLBACK
```

反向查询

下面是反向查询的示例，反向查询是指从没有relationship逻辑字段的表开始查询：

In [9]:

```python
# 查询所有班级中的所有学生，学生表中有relationship，并且它的backref为have_student，所以我们可以
# 另外，对于班级来说，学生可以有多个，所以have_student应当是一个序列
classes_lst = session.query(
    ClassesInfo
).all()
for row in classes_lst:
    print("class name :", row.name)
    for student in row.have_student:
        print("student name :", student.name)
# class name : one year one class
#      student name : Jack
#      student name : Anna
# class name : one year two class
```

```
#      student name : Tom
# class name : one year three class
#      student name : Mary
#      student name : Bobby
# 关闭链接，亦可使用session.remove()，它将回收该链接
session.close()
```

```
2026-05-07 13:15:05,373 INFO sqlalchemy.engine.Engine BEGIN (implicit)
2026-05-07 13:15:05,375 INFO sqlalchemy.engine.Engine SELECT "classesInfo".id AS "classesInfo_
id", "classesInfo".number AS "classesInfo_number", "classesInfo".name AS "classesInfo_name",
"classesInfo".fk_teacher_id AS "classesInfo_fk_teacher_id"
FROM "classesInfo"
2026-05-07 13:15:05,376 INFO sqlalchemy.engine.Engine [generated in 0.00072s] {}
class name : one year one class
2026-05-07 13:15:05,391 INFO sqlalchemy.engine.Engine SELECT "studentsInfo".id AS "studentsInf
o_id", "studentsInfo".name AS "studentsInfo_name", "studentsInfo".gender AS "studentsInfo_gend
er", "studentsInfo".age AS "studentsInfo_age", "studentsInfo".fk_student_id AS "studentsInfo_f
k_student_id", "studentsInfo".fk_class_id AS "studentsInfo_fk_class_id"
FROM "studentsInfo"
WHERE %(param_1)s = "studentsInfo".fk_class_id
2026-05-07 13:15:05,392 INFO sqlalchemy.engine.Engine [generated in 0.00109s] {'param_1': 1}
student name : Tom
student name : Anna
class name : one year two class
2026-05-07 13:15:05,401 INFO sqlalchemy.engine.Engine SELECT "studentsInfo".id AS "studentsInf
o_id", "studentsInfo".name AS "studentsInfo_name", "studentsInfo".gender AS "studentsInfo_gend
er", "studentsInfo".age AS "studentsInfo_age", "studentsInfo".fk_student_id AS "studentsInfo_f
k_student_id", "studentsInfo".fk_class_id AS "studentsInfo_fk_class_id"
FROM "studentsInfo"
WHERE %(param_1)s = "studentsInfo".fk_class_id
2026-05-07 13:15:05,402 INFO sqlalchemy.engine.Engine [cached since 0.01022s ago] {'param_1':
2}
student name : Jack
student name : Bobby
class name : one year three class
2026-05-07 13:15:05,410 INFO sqlalchemy.engine.Engine SELECT "studentsInfo".id AS "studentsInf
o_id", "studentsInfo".name AS "studentsInfo_name", "studentsInfo".gender AS "studentsInfo_gend
er", "studentsInfo".age AS "studentsInfo_age", "studentsInfo".fk_student_id AS "studentsInfo_f
k_student_id", "studentsInfo".fk_class_id AS "studentsInfo_fk_class_id"
FROM "studentsInfo"
WHERE %(param_1)s = "studentsInfo".fk_class_id
2026-05-07 13:15:05,411 INFO sqlalchemy.engine.Engine [cached since 0.01926s ago] {'param_1':
3}
student name : Mary
2026-05-07 13:15:05,418 INFO sqlalchemy.engine.Engine ROLLBACK
```

总结，正向查询的逻辑字段总是得到一个对象，反向查询的逻辑字段总是得到一个列表。

反向方法

使用逻辑字段relationship可以直接对一些跨表记录进行增删改查。

由于逻辑字段是一个类似于列表的存在（仅限于反向查询，正向查询总是得到一个对象），所以列表的绝大多数方法都能用。

<class 'sqlalchemy.orm.collections.InstrumentedList'> - append() - clear() - copy() - count() - extend() - index() - insert() - pop() - remove() - reverse() - sort() 下面不再进行实机演示，因为我们上面的几张表中做了很多约束。

In [10]:
```
# 一下代码只是举例，运行时不会成功


# 比如
# 给老师增加班级
```

```
result = session.query(Teachers).first()
# extend方法：
result.re_class.extend([
    Classes(name="三年级一班",),
    Classes(name="三年级二班",),
])
# 比如
# 减少老师所在的班级
result = session.query(Teachers).first()
# 待删除的班级对象,集合查找比较快
delete_class_set = {
    session.query(Classes).filter_by(id=7).first(),
    session.query(Classes).filter_by(id=8).first(),
}
# 循换老师所在的班级
# remove方法：
for class_obj in result.re_class:
    if class_obj in delete_class_set:
        result.re_class.remove(class_obj)
# 比如
# 清空老师所任教的所有班级
# 拿出一个老师
result = session.query(Teachers).first()
result.re_class.clear()
```

```
---------------------------------------------------------------------------
NameError                                 Traceback (most recent call last)
Cell In[10], line 6
      1 # 一下代码只是举例，运行时不会成功
      2
      3
      4 # 比如
      5 # 给老师增加班级
----> 6 result = session.query(Teachers).first()
      7 # extend方法：
      8 result.re_class.extend([
      9     Classes(name="三年级一班",),
     10     Classes(name="三年级二班",),
     11 ])

NameError: name 'Teachers' is not defined
```

查询案例

（1）查看每个班级共有多少学生：

In [11]:

```
#JOIN查询：

# 获取链接池、ORM表对象
from sqlalchemy import func
result = session.query(
    ClassesInfo.name,
    func.count(StudentsInfo.id)
).join(
    StudentsInfo,
    ClassesInfo.id == StudentsInfo.fk_class_id
).group_by(
    ClassesInfo.id
).all()
print(result)
# [('one year one class', 2), ('one year two class', 2), ('one year three class', 1)]
# 关闭链接，亦可使用session.remove()，它将回收该链接
session.close()
```

```
2026-05-07 13:17:31,447 INFO sqlalchemy.engine.Engine BEGIN (implicit)
2026-05-07 13:17:31,452 INFO sqlalchemy.engine.Engine SELECT "classesInfo".name AS "classesInf
o_name", count("studentsInfo".id) AS count_1
FROM "classesInfo" JOIN "studentsInfo" ON "classesInfo".id = "studentsInfo".fk_class_id GROUP
BY "classesInfo".id
2026-05-07 13:17:31,453 INFO sqlalchemy.engine.Engine [generated in 0.00109s] {}
[('one year two class', 2), ('one year one class', 2), ('one year three class', 1)]
2026-05-07 13:17:31,471 INFO sqlalchemy.engine.Engine ROLLBACK
```

In [12]:

```python
#正反查询:
result = {}
class_lst = session.query(
    ClassesInfo
).all()
for row in class_lst:
    for student in row.have_student:
        count = result.setdefault(row.name, 0)
        result[row.name] = count + 1
print(result.items())
# dict_items([('one year one class', 2), ('one year two class', 2), ('one year three class', 1
# 关闭链接，亦可使用session.remove()，它将回收该链接
session.close()
```

```
2026-05-07 13:17:36,719 INFO sqlalchemy.engine.Engine BEGIN (implicit)
2026-05-07 13:17:36,721 INFO sqlalchemy.engine.Engine SELECT "classesInfo".id AS "classesInfo_
id", "classesInfo".number AS "classesInfo_number", "classesInfo".name AS "classesInfo_name",
"classesInfo".fk_teacher_id AS "classesInfo_fk_teacher_id"
FROM "classesInfo"
2026-05-07 13:17:36,721 INFO sqlalchemy.engine.Engine [cached since 151.3s ago] {}
2026-05-07 13:17:36,742 INFO sqlalchemy.engine.Engine SELECT "studentsInfo".id AS "studentsInf
o_id", "studentsInfo".name AS "studentsInfo_name", "studentsInfo".gender AS "studentsInfo_gend
er", "studentsInfo".age AS "studentsInfo_age", "studentsInfo".fk_student_id AS "studentsInfo_f
k_student_id", "studentsInfo".fk_class_id AS "studentsInfo_fk_class_id"
FROM "studentsInfo"
WHERE %(param_1)s = "studentsInfo".fk_class_id
2026-05-07 13:17:36,743 INFO sqlalchemy.engine.Engine [cached since 151.4s ago] {'param_1': 1}
2026-05-07 13:17:36,753 INFO sqlalchemy.engine.Engine SELECT "studentsInfo".id AS "studentsInf
o_id", "studentsInfo".name AS "studentsInfo_name", "studentsInfo".gender AS "studentsInfo_gend
er", "studentsInfo".age AS "studentsInfo_age", "studentsInfo".fk_student_id AS "studentsInfo_f
k_student_id", "studentsInfo".fk_class_id AS "studentsInfo_fk_class_id"
FROM "studentsInfo"
WHERE %(param_1)s = "studentsInfo".fk_class_id
2026-05-07 13:17:36,754 INFO sqlalchemy.engine.Engine [cached since 151.4s ago] {'param_1': 2}
2026-05-07 13:17:36,765 INFO sqlalchemy.engine.Engine SELECT "studentsInfo".id AS "studentsInf
o_id", "studentsInfo".name AS "studentsInfo_name", "studentsInfo".gender AS "studentsInfo_gend
er", "studentsInfo".age AS "studentsInfo_age", "studentsInfo".fk_student_id AS "studentsInfo_f
k_student_id", "studentsInfo".fk_class_id AS "studentsInfo_fk_class_id"
FROM "studentsInfo"
WHERE %(param_1)s = "studentsInfo".fk_class_id
2026-05-07 13:17:36,765 INFO sqlalchemy.engine.Engine [cached since 151.4s ago] {'param_1': 3}
dict_items([('one year one class', 2), ('one year two class', 2), ('one year three class',
1)])
2026-05-07 13:17:36,775 INFO sqlalchemy.engine.Engine ROLLBACK
```

(2) 查看每个学生的入学、毕业年份以及所在的班级名称:

In [13]:

```python
#JOIN查询:
result = session.query(
    StudentsNumberInfo.number,
    StudentsInfo.name,
    ClassesInfo.name,
    StudentsNumberInfo.admission,
    StudentsNumberInfo.graduation
).join(
    StudentsInfo,
    StudentsInfo.fk_class_id == ClassesInfo.id
```

```
).join(
    StudentsNumberInfo,
    StudentsNumberInfo.id == StudentsInfo.fk_student_id
).order_by(
    StudentsNumberInfo.number.asc()
).all()
print(result)
# [
#     (160101, 'Tom', 'one year one class', datetime.date(2016, 9, 1), datetime.date(2021, 6,
#     (160102, 'Anna', 'one year one class', datetime.date(2016, 9, 1), datetime.date(2021, 6
#     (160201, 'Jack', 'one year two class', datetime.date(2016, 9, 1), datetime.date(2021, 6
#     (160202, 'Bobby', 'one year two class', datetime.date(2016, 9, 1), datetime.date(2021, 
#     (160301, 'Mary', 'one year three class', datetime.date(2016, 9, 1), datetime.date(2021,
# ]
# 关闭链接，亦可使用session.remove()，它将回收该链接
session.close()
```

```
2026-05-07 13:17:39,187 INFO sqlalchemy.engine.Engine BEGIN (implicit)
2026-05-07 13:17:39,190 INFO sqlalchemy.engine.Engine SELECT "studentsNumberInfo".number AS "s
tudentsNumberInfo_number", "studentsInfo".name AS "studentsInfo_name", "classesInfo".name AS
"classesInfo_name", "studentsNumberInfo".admission AS "studentsNumberInfo_admission", "student
sNumberInfo".graduation AS "studentsNumberInfo_graduation"
FROM "classesInfo" JOIN "studentsInfo" ON "studentsInfo".fk_class_id = "classesInfo".id JOIN
"studentsNumberInfo" ON "studentsNumberInfo".id = "studentsInfo".fk_student_id ORDER BY "stude
ntsNumberInfo".number ASC
2026-05-07 13:17:39,190 INFO sqlalchemy.engine.Engine [generated in 0.00132s] {}
[(160101, 'Tom', 'one year one class', datetime.date(2016, 9, 1), datetime.date(2021, 6, 15)),
(160102, 'Anna', 'one year one class', datetime.date(2016, 9, 1), datetime.date(2021, 6, 15)),
(160201, 'Jack', 'one year two class', datetime.date(2016, 9, 1), datetime.date(2021, 6, 15)),
(160202, 'Bobby', 'one year two class', datetime.date(2016, 9, 1), datetime.date(2021, 6, 1
5)), (160301, 'Mary', 'one year three class', datetime.date(2016, 9, 1), datetime.date(2021,
6, 15))]
2026-05-07 13:17:39,214 INFO sqlalchemy.engine.Engine ROLLBACK
```

In [14]:

```
#正反查询：
result = []
student_lst = session.query(
    StudentsInfo
).all()
for row in student_lst:
    result.append((
        row.number_info.number,
        row.name,
        row.from_class.name,
        row.number_info.admission,
        row.number_info.graduation
    ))
print(result)
# [
#     (160101, 'Tom', 'one year one class', datetime.date(2016, 9, 1), datetime.date(2021, 6,
#     (160102, 'Anna', 'one year one class', datetime.date(2016, 9, 1), datetime.date(2021, 6
#     (160201, 'Jack', 'one year two class', datetime.date(2016, 9, 1), datetime.date(2021, 6
#     (160202, 'Bobby', 'one year two class', datetime.date(2016, 9, 1), datetime.date(2021, 
#     (160301, 'Mary', 'one year three class', datetime.date(2016, 9, 1), datetime.date(2021,
# ]
# 关闭链接，亦可使用session.remove()，它将回收该链接
session.close()
```

```
2026-05-07 13:17:41,365 INFO sqlalchemy.engine.Engine BEGIN (implicit)
2026-05-07 13:17:41,366 INFO sqlalchemy.engine.Engine SELECT "studentsInfo".id AS "studentsInf
o_id", "studentsInfo".name AS "studentsInfo_name", "studentsInfo".gender AS "studentsInfo_gend
er", "studentsInfo".age AS "studentsInfo_age", "studentsInfo".fk_student_id AS "studentsInfo_f
k_student_id", "studentsInfo".fk_class_id AS "studentsInfo_fk_class_id"
FROM "studentsInfo"
2026-05-07 13:17:41,366 INFO sqlalchemy.engine.Engine [cached since 161.4s ago] {}
2026-05-07 13:17:41,384 INFO sqlalchemy.engine.Engine SELECT "studentsNumberInfo".id AS "stude
ntsNumberInfo_id", "studentsNumberInfo".number AS "studentsNumberInfo_number", "studentsNumber
Info".admission AS "studentsNumberInfo_admission", "studentsNumberInfo".graduation AS "student
sNumberInfo_graduation"
FROM "studentsNumberInfo"
WHERE "studentsNumberInfo".id = %(pk_1)s
2026-05-07 13:17:41,385 INFO sqlalchemy.engine.Engine [generated in 0.00121s] {'pk_1': 1}
2026-05-07 13:17:41,394 INFO sqlalchemy.engine.Engine SELECT "classesInfo".id AS "classesInfo_
id", "classesInfo".number AS "classesInfo_number", "classesInfo".name AS "classesInfo_name",
"classesInfo".fk_teacher_id AS "classesInfo_fk_teacher_id"
FROM "classesInfo"
WHERE "classesInfo".id = %(pk_1)s
2026-05-07 13:17:41,395 INFO sqlalchemy.engine.Engine [cached since 161.4s ago] {'pk_1': 2}
2026-05-07 13:17:41,407 INFO sqlalchemy.engine.Engine SELECT "studentsNumberInfo".id AS "stude
ntsNumberInfo_id", "studentsNumberInfo".number AS "studentsNumberInfo_number", "studentsNumber
Info".admission AS "studentsNumberInfo_admission", "studentsNumberInfo".graduation AS "student
sNumberInfo_graduation"
FROM "studentsNumberInfo"
WHERE "studentsNumberInfo".id = %(pk_1)s
2026-05-07 13:17:41,409 INFO sqlalchemy.engine.Engine [cached since 0.02554s ago] {'pk_1': 2}
2026-05-07 13:17:41,418 INFO sqlalchemy.engine.Engine SELECT "classesInfo".id AS "classesInfo_
id", "classesInfo".number AS "classesInfo_number", "classesInfo".name AS "classesInfo_name",
"classesInfo".fk_teacher_id AS "classesInfo_fk_teacher_id"
FROM "classesInfo"
WHERE "classesInfo".id = %(pk_1)s
2026-05-07 13:17:41,419 INFO sqlalchemy.engine.Engine [cached since 161.4s ago] {'pk_1': 1}
2026-05-07 13:17:41,428 INFO sqlalchemy.engine.Engine SELECT "studentsNumberInfo".id AS "stude
ntsNumberInfo_id", "studentsNumberInfo".number AS "studentsNumberInfo_number", "studentsNumber
Info".admission AS "studentsNumberInfo_admission", "studentsNumberInfo".graduation AS "student
sNumberInfo_graduation"
FROM "studentsNumberInfo"
WHERE "studentsNumberInfo".id = %(pk_1)s
2026-05-07 13:17:41,429 INFO sqlalchemy.engine.Engine [cached since 0.04606s ago] {'pk_1': 3}
2026-05-07 13:17:41,441 INFO sqlalchemy.engine.Engine SELECT "classesInfo".id AS "classesInfo_
id", "classesInfo".number AS "classesInfo_number", "classesInfo".name AS "classesInfo_name",
"classesInfo".fk_teacher_id AS "classesInfo_fk_teacher_id"
FROM "classesInfo"
WHERE "classesInfo".id = %(pk_1)s
2026-05-07 13:17:41,442 INFO sqlalchemy.engine.Engine [cached since 161.4s ago] {'pk_1': 3}
2026-05-07 13:17:41,454 INFO sqlalchemy.engine.Engine SELECT "studentsNumberInfo".id AS "stude
ntsNumberInfo_id", "studentsNumberInfo".number AS "studentsNumberInfo_number", "studentsNumber
Info".admission AS "studentsNumberInfo_admission", "studentsNumberInfo".graduation AS "student
sNumberInfo_graduation"
FROM "studentsNumberInfo"
WHERE "studentsNumberInfo".id = %(pk_1)s
2026-05-07 13:17:41,455 INFO sqlalchemy.engine.Engine [cached since 0.07161s ago] {'pk_1': 4}
2026-05-07 13:17:41,463 INFO sqlalchemy.engine.Engine SELECT "studentsNumberInfo".id AS "stude
ntsNumberInfo_id", "studentsNumberInfo".number AS "studentsNumberInfo_number", "studentsNumber
Info".admission AS "studentsNumberInfo_admission", "studentsNumberInfo".graduation AS "student
sNumberInfo_graduation"
FROM "studentsNumberInfo"
WHERE "studentsNumberInfo".id = %(pk_1)s
2026-05-07 13:17:41,464 INFO sqlalchemy.engine.Engine [cached since 0.08093s ago] {'pk_1': 6}
[(160201, 'Jack', 'one year two class', datetime.date(2016, 9, 1), datetime.date(2021, 6, 1
5)), (160101, 'Tom', 'one year one class', datetime.date(2016, 9, 1), datetime.date(2021, 6, 1
5)), (160301, 'Mary', 'one year three class', datetime.date(2016, 9, 1), datetime.date(2021,
6, 15)), (160102, 'Anna', 'one year one class', datetime.date(2016, 9, 1), datetime.date(2021,
6, 15)), (160202, 'Bobby', 'one year two class', datetime.date(2016, 9, 1), datetime.date(202
```

```
1, 6, 15))]
2026-05-07 13:17:41,477 INFO sqlalchemy.engine.Engine ROLLBACK
```

3）查看David所教授的学生中年龄最小的学生：

练习1：

In [15]:
```python
# todo
#JOIN查询
# [('David', 'Mary', 16, 'one year three class')]
result = session.query(
    TeachersInfo.name,
    StudentsInfo.name,
    StudentsInfo.age,
    ClassesInfo.name
).select_from(TeachersInfo).join(
    ClassesAndTeachersRelationship, ClassesAndTeachersRelationship.fk_teacher_id == TeachersI
).join(
    ClassesInfo, ClassesInfo.id == ClassesAndTeachersRelationship.fk_class_id
).join(
    StudentsInfo, StudentsInfo.fk_class_id == ClassesInfo.id
).filter(
    TeachersInfo.name == "David"
).order_by(
    StudentsInfo.age.asc()
).limit(1).all()
print(result)
```

```
2026-05-07 13:22:52,143 INFO sqlalchemy.engine.Engine BEGIN (implicit)
2026-05-07 13:22:52,145 INFO sqlalchemy.engine.Engine SELECT "teachersInfo".name AS "teachersI
nfo_name", "studentsInfo".name AS "studentsInfo_name", "studentsInfo".age AS "studentsInfo_ag
e", "classesInfo".name AS "classesInfo_name"
FROM "teachersInfo" JOIN "classesAndTeachersRelationship" ON "classesAndTeachersRelationship".
fk_teacher_id = "teachersInfo".id JOIN "classesInfo" ON "classesInfo".id = "classesAndTeachers
Relationship".fk_class_id JOIN "studentsInfo" ON "studentsInfo".fk_class_id = "classesInfo".id
WHERE "teachersInfo".name = %(name_1)s ORDER BY "studentsInfo".age ASC
 LIMIT %(param_1)s
2026-05-07 13:22:52,147 INFO sqlalchemy.engine.Engine [generated in 0.00204s] {'name_1': 'Davi
d', 'param_1': 1}
[('David', 'Mary', 16, 'one year three class')]
```

练习2:

In [16]:
```python
#正反查询:
# ('David', 'Mary', 16, 'one year three class')
david = session.query(TeachersInfo).filter_by(name="David").first()
res_list = []
# david.mid 拿到所有中间表记录，再通过 mid_to_class 拿到班级，再取班级里的学生
for rel in david.mid:
    cls_obj = rel.mid_to_class
    for stu in cls_obj.have_student:
        res_list.append((david.name, stu.name, stu.age, cls_obj.name))
# 纯 Python 逻辑按照年龄排序取最小的一个
result = min(res_list, key=lambda x: x[2])
print(result)
```

```
2026-05-07 13:23:09,739 INFO sqlalchemy.engine.Engine SELECT "teachersInfo".id AS "teachersInf
o_id", "teachersInfo".number AS "teachersInfo_number", "teachersInfo".name AS "teachersInfo_na
me", "teachersInfo".gender AS "teachersInfo_gender", "teachersInfo".age AS "teachersInfo_age"
FROM "teachersInfo"
WHERE "teachersInfo".name = %(name_1)s
 LIMIT %(param_1)s
2026-05-07 13:23:09,740 INFO sqlalchemy.engine.Engine [generated in 0.00074s] {'name_1': 'Davi
d', 'param_1': 1}
2026-05-07 13:23:09,754 INFO sqlalchemy.engine.Engine SELECT "classesAndTeachersRelationship".
id AS "classesAndTeachersRelationship_id", "classesAndTeachersRelationship".fk_teacher_id AS
"classesAndTeachersRelationship_fk_teacher_id", "classesAndTeachersRelationship".fk_class_id A
S "classesAndTeachersRelationship_fk_class_id"
FROM "classesAndTeachersRelationship"
WHERE %(param_1)s = "classesAndTeachersRelationship".fk_teacher_id
2026-05-07 13:23:09,755 INFO sqlalchemy.engine.Engine [generated in 0.00092s] {'param_1': 1}
2026-05-07 13:23:09,766 INFO sqlalchemy.engine.Engine SELECT "classesInfo".id AS "classesInfo_
id", "classesInfo".number AS "classesInfo_number", "classesInfo".name AS "classesInfo_name",
"classesInfo".fk_teacher_id AS "classesInfo_fk_teacher_id"
FROM "classesInfo"
WHERE "classesInfo".id = %(pk_1)s
2026-05-07 13:23:09,767 INFO sqlalchemy.engine.Engine [generated in 0.00148s] {'pk_1': 1}
2026-05-07 13:23:09,776 INFO sqlalchemy.engine.Engine SELECT "studentsInfo".id AS "studentsInf
o_id", "studentsInfo".name AS "studentsInfo_name", "studentsInfo".gender AS "studentsInfo_gend
er", "studentsInfo".age AS "studentsInfo_age", "studentsInfo".fk_student_id AS "studentsInfo_f
k_student_id", "studentsInfo".fk_class_id AS "studentsInfo_fk_class_id"
FROM "studentsInfo"
WHERE %(param_1)s = "studentsInfo".fk_class_id
2026-05-07 13:23:09,777 INFO sqlalchemy.engine.Engine [cached since 484.4s ago] {'param_1': 1}
2026-05-07 13:23:09,787 INFO sqlalchemy.engine.Engine SELECT "classesInfo".id AS "classesInfo_
id", "classesInfo".number AS "classesInfo_number", "classesInfo".name AS "classesInfo_name",
"classesInfo".fk_teacher_id AS "classesInfo_fk_teacher_id"
FROM "classesInfo"
WHERE "classesInfo".id = %(pk_1)s
2026-05-07 13:23:09,788 INFO sqlalchemy.engine.Engine [cached since 0.02333s ago] {'pk_1': 2}
2026-05-07 13:23:09,798 INFO sqlalchemy.engine.Engine SELECT "studentsInfo".id AS "studentsInf
o_id", "studentsInfo".name AS "studentsInfo_name", "studentsInfo".gender AS "studentsInfo_gend
er", "studentsInfo".age AS "studentsInfo_age", "studentsInfo".fk_student_id AS "studentsInfo_f
k_student_id", "studentsInfo".fk_class_id AS "studentsInfo_fk_class_id"
FROM "studentsInfo"
WHERE %(param_1)s = "studentsInfo".fk_class_id
2026-05-07 13:23:09,800 INFO sqlalchemy.engine.Engine [cached since 484.4s ago] {'param_1': 2}
2026-05-07 13:23:09,809 INFO sqlalchemy.engine.Engine SELECT "classesInfo".id AS "classesInfo_
id", "classesInfo".number AS "classesInfo_number", "classesInfo".name AS "classesInfo_name",
"classesInfo".fk_teacher_id AS "classesInfo_fk_teacher_id"
FROM "classesInfo"
WHERE "classesInfo".id = %(pk_1)s
2026-05-07 13:23:09,810 INFO sqlalchemy.engine.Engine [cached since 0.04426s ago] {'pk_1': 3}
2026-05-07 13:23:09,819 INFO sqlalchemy.engine.Engine SELECT "studentsInfo".id AS "studentsInf
o_id", "studentsInfo".name AS "studentsInfo_name", "studentsInfo".gender AS "studentsInfo_gend
er", "studentsInfo".age AS "studentsInfo_age", "studentsInfo".fk_student_id AS "studentsInfo_f
k_student_id", "studentsInfo".fk_class_id AS "studentsInfo_fk_class_id"
FROM "studentsInfo"
WHERE %(param_1)s = "studentsInfo".fk_class_id
2026-05-07 13:23:09,821 INFO sqlalchemy.engine.Engine [cached since 484.4s ago] {'param_1': 3}
('David', 'Mary', 16, 'one year three class')
```

4）查看每个班级的负责人是谁，以及任课老师都有谁：

练习3:

In [17]:

```python
# JOIN
# todo
# [('one year one class', 'David', 'Jason,David'), ('one year two class', 'Jason', 'David'),

from sqlalchemy.orm import aliased
```

```
from sqlalchemy import func

Leader = aliased(TeachersInfo)
Teacher = aliased(TeachersInfo)

result = session.query(
    ClassesInfo.name,
    Leader.name,
    func.string_agg(Teacher.name, ',')
).select_from(ClassesInfo).join(
    Leader, ClassesInfo.fk_teacher_id == Leader.id
).join(
    ClassesAndTeachersRelationship, ClassesInfo.id == ClassesAndTeachersRelationship.fk_class
).join(
    Teacher, ClassesAndTeachersRelationship.fk_teacher_id == Teacher.id
).group_by(
    ClassesInfo.name, Leader.name
).all()

print(result)
```

```
2026-05-07 13:23:35,803 INFO sqlalchemy.engine.Engine SELECT "classesInfo".name AS "classesInf
o_name", "teachersInfo_1".name AS "teachersInfo_1_name", string_agg("teachersInfo_2".name, %(s
tring_agg_2)s) AS string_agg_1
FROM "classesInfo" JOIN "teachersInfo" AS "teachersInfo_1" ON "classesInfo".fk_teacher_id = "t
eachersInfo_1".id JOIN "classesAndTeachersRelationship" ON "classesInfo".id = "classesAndTeach
ersRelationship".fk_class_id JOIN "teachersInfo" AS "teachersInfo_2" ON "classesAndTeachersRel
ationship".fk_teacher_id = "teachersInfo_2".id GROUP BY "classesInfo".name, "teachersInfo_1".n
ame
2026-05-07 13:23:35,804 INFO sqlalchemy.engine.Engine [generated in 0.00158s] {'string_agg_2':
','}
[('one year one class', 'David', 'David,Jason'), ('one year three class', 'Lisa', 'David,Lis
a'), ('one year two class', 'Jason', 'David')]
```

练习4

In [18]:

```
# 正反查询
# todo
# [('one year one class', 'David', 'Jason,David'), ('one year two class', 'Jason', 'David'),

result = []
classes = session.query(ClassesInfo).all()

for c in classes:
    # 班级负责人 (一对一)
    leader_name = c.leader_teacher.name

    # 遍历该班级在关联表中的记录，取出任课教师名字拼接
    teachers = [rel.mid_to_teacher.name for rel in c.mid]

    result.append((c.name, leader_name, ",".join(teachers)))

print(result)
```

```
2026-05-07 13:24:26,401 INFO sqlalchemy.engine.Engine SELECT "classesInfo".id AS "classesInfo_
id", "classesInfo".number AS "classesInfo_number", "classesInfo".name AS "classesInfo_name",
"classesInfo".fk_teacher_id AS "classesInfo_fk_teacher_id"
FROM "classesInfo"
2026-05-07 13:24:26,402 INFO sqlalchemy.engine.Engine [cached since 561s ago] {}
2026-05-07 13:24:26,414 INFO sqlalchemy.engine.Engine SELECT "classesAndTeachersRelationship".
id AS "classesAndTeachersRelationship_id", "classesAndTeachersRelationship".fk_teacher_id AS
"classesAndTeachersRelationship_fk_teacher_id", "classesAndTeachersRelationship".fk_class_id A
S "classesAndTeachersRelationship_fk_class_id"
FROM "classesAndTeachersRelationship"
WHERE %(param_1)s = "classesAndTeachersRelationship".fk_class_id
2026-05-07 13:24:26,415 INFO sqlalchemy.engine.Engine [generated in 0.00094s] {'param_1': 1}
2026-05-07 13:24:26,424 INFO sqlalchemy.engine.Engine SELECT "teachersInfo".id AS "teachersInf
o_id", "teachersInfo".number AS "teachersInfo_number", "teachersInfo".name AS "teachersInfo_na
me", "teachersInfo".gender AS "teachersInfo_gender", "teachersInfo".age AS "teachersInfo_age"
FROM "teachersInfo"
WHERE "teachersInfo".id = %(pk_1)s
2026-05-07 13:24:26,426 INFO sqlalchemy.engine.Engine [generated in 0.00159s] {'pk_1': 2}
2026-05-07 13:24:26,435 INFO sqlalchemy.engine.Engine SELECT "classesAndTeachersRelationship".
id AS "classesAndTeachersRelationship_id", "classesAndTeachersRelationship".fk_teacher_id AS
"classesAndTeachersRelationship_fk_teacher_id", "classesAndTeachersRelationship".fk_class_id A
S "classesAndTeachersRelationship_fk_class_id"
FROM "classesAndTeachersRelationship"
WHERE %(param_1)s = "classesAndTeachersRelationship".fk_class_id
2026-05-07 13:24:26,436 INFO sqlalchemy.engine.Engine [cached since 0.02251s ago] {'param_1':
2}
2026-05-07 13:24:26,445 INFO sqlalchemy.engine.Engine SELECT "teachersInfo".id AS "teachersInf
o_id", "teachersInfo".number AS "teachersInfo_number", "teachersInfo".name AS "teachersInfo_na
me", "teachersInfo".gender AS "teachersInfo_gender", "teachersInfo".age AS "teachersInfo_age"
FROM "teachersInfo"
WHERE "teachersInfo".id = %(pk_1)s
2026-05-07 13:24:26,446 INFO sqlalchemy.engine.Engine [generated in 0.00117s] {'pk_1': 3}
2026-05-07 13:24:26,456 INFO sqlalchemy.engine.Engine SELECT "classesAndTeachersRelationship".
id AS "classesAndTeachersRelationship_id", "classesAndTeachersRelationship".fk_teacher_id AS
"classesAndTeachersRelationship_fk_teacher_id", "classesAndTeachersRelationship".fk_class_id A
S "classesAndTeachersRelationship_fk_class_id"
FROM "classesAndTeachersRelationship"
WHERE %(param_1)s = "classesAndTeachersRelationship".fk_class_id
2026-05-07 13:24:26,457 INFO sqlalchemy.engine.Engine [cached since 0.04328s ago] {'param_1':
3}
[('one year one class', 'David', 'David,Jason'), ('one year two class', 'Jason', 'David'), ('o
ne year three class', 'Lisa', 'David,Lisa')]
```

原生SQL

查看执行命令

如果一条查询语句是filter()结尾，则该对象的__str__方法会返回格式化后的查询语句：

In [19]:
```
print(
    session.query(StudentsInfo).filter()
)
```

```
SELECT "studentsInfo".id AS "studentsInfo_id", "studentsInfo".name AS "studentsInfo_name", "st
udentsInfo".gender AS "studentsInfo_gender", "studentsInfo".age AS "studentsInfo_age", "studen
tsInfo".fk_student_id AS "studentsInfo_fk_student_id", "studentsInfo".fk_class_id AS "students
Info_fk_class_id"
FROM "studentsInfo"
```